CS 216: Bitcoin Transaction Lab

Team Name	HighOnByte
Course	CS 216
Assignment	2nd Programming Assignment
Deadline	10th March 2026

Team Members

Name	Roll Number
Aastha	240004001
Bhavika Jaiswal	240001017
Sanskriti Jain	240001064
Suhani	240001077

GitHub Repository:
https://github.com/Sanskriti-hello/bitcoin-transaction-lab

Contents

1 Theoretical Background **2**
1.1 Bitcoin Script . . . 2
1.2 P2PKH – Pay to Public Key Hash (Legacy) . . . 2
1.3 P2SH-P2WPKH – SegWit wrapped in P2SH . . . 2

2 Part 1: Legacy P2PKH Transactions **4**
2.1 Bitcoin Core Configuration . . . 4
2.2 Generated Legacy Addresses . . . 4
2.3 Transaction Workflow: A to B to C . . . 4
2.4 Decoded Scripts – Transaction A to B . . . 4
2.5 Decoded Scripts – Transaction B to C . . . 5
2.6 Transaction Sizes – P2PKH . . . 5

3 Part 1: btcdeb Script Execution (P2PKH) **6**
3.1 Command Used . . . 6
3.2 Initial Script Load . . . 6
3.3 Step-by-Step Execution . . . 6
3.4 Final Result . . . 8

4 Part 2: P2SH-P2WPKH SegWit Transactions **9**
4.1 Generated SegWit Addresses . . . 9
4.2 Transaction Workflow: A' to B' to C' . . . 9
4.3 Decoded Scripts – Transaction A' to B' . . . 9
4.4 Decoded Scripts – Transaction B' to C' . . . 9
4.5 Transaction Sizes – SegWit . . . 10

5 Part 2: btcdeb Script Execution (SegWit) **11**
5.1 Command Used . . . 11
5.2 Startup Output . . . 11
5.3 Script Loaded . . . 11
5.4 Step-by-Step Execution . . . 12
5.5 Final Result . . . 13

6 Part 3: Comparative Analysis **14**
6.1 Transaction Size Comparison . . . 14
6.2 Script Structure Comparison . . . 14
6.3 Weight Calculation – Worked Example . . . 14
6.4 Why SegWit Transactions Are Smaller . . . 14

7 Conclusion **16**

1 Theoretical Background

1.1 Bitcoin Script

Bitcoin Script is a simple, stack-based programming language used to define the conditions under which Bitcoin can be spent. It has no loops and is not Turing-complete, making it simple and safe to execute. Transaction validation works by combining two scripts:

- **scriptSig** (unlocking script) – provided by the spender in the input
- **scriptPubKey** (locking script) – stored in the UTXO being spent

They are concatenated and executed together. The transaction is valid only if the final stack value is `TRUE (0x01)`.

1.2 P2PKH – Pay to Public Key Hash (Legacy)

P2PKH is the classic Bitcoin address format. In regtest mode, addresses start with `m` or `n`.

Address derivation:

```
Private Key -> Public Key -> SHA256 -> RIPEMD160 -> Bitcoin Address
```

Challenge Script (scriptPubKey) – 25 bytes

```
OP_DUP OP_HASH160 <pubKeyHash_20bytes> OP_EQUALVERIFY OP_CHECKSIG
hex prefix: 76a914...88ac
```

Response Script (scriptSig) – 107 bytes

```
<DER_signature ~72bytes> <compressed_pubkey_33bytes>
```

Stack Execution Trace

Step	Operation	Stack State
0	Push signature (from scriptSig)	[sig]
1	Push pubkey (from scriptSig)	[sig, pubKey]
2	OP_DUP	[sig, pubKey, pubKey]
3	OP_HASH160	[sig, pubKey, hash160(pubKey)]
4	Push expected hash (from scriptPubKey)	[sig, pubKey, computed, expected]
5	OP_EQUALVERIFY – must match	[sig, pubKey]
6	OP_CHECKSIG – verify ECDSA sig	[TRUE / 0x01]

1.3 P2SH-P2WPKH – SegWit wrapped in P2SH

SegWit (BIP 141) separates signature data into a **witness** field. In P2SH-P2WPKH, a native SegWit script is wrapped inside a P2SH address for backward compatibility. Addresses start with `2` in regtest.

scriptPubKey (on-chain) – 23 bytes

```
OP_HASH160 <hash160(redeemScript)> OP_EQUAL
hex prefix: a914...87
```

redeemScript (revealed when spending) – 22 bytes

```
OP_0 <hash160(publicKey)>
hex: 0014<20bytes>
```

scriptSig – 23 bytes only

```
<serialized redeemScript>   <- only the redeem script, NOT the signature
```

Witness (segregated, weight discounted)

```
[0]: <DER_signature ~72bytes>
[1]: <compressed_pubkey 33bytes>
```

Weight Calculation

```
weight = (non-witness bytes x 4) + (witness bytes x 1)
vbytes = ceil(weight / 4)

Witness bytes get 75% discount compared to regular bytes.
```

2 Part 1: Legacy P2PKH Transactions

2.1 Bitcoin Core Configuration

`bitcoind` was run in regtest mode with the following `bitcoin.conf`:

```
regtest=1
server=1
txindex=1

[regtest]
rpcuser=student
rpcpassword=cs216bitcoin
rpcport=18443
rpcbind=127.0.0.1
rpcallowip=127.0.0.1
paytxfee=0.0001
fallbackfee=0.0002
mintxfee=0.00001
txconfirmtarget=6
```

2.2 Generated Legacy Addresses

Three legacy (P2PKH) addresses were generated using `getnewaddress "label" "legacy"`:

Label	Address
Address A	`n2WRkpMzvpQ5UAM34o1JtkDTwCMW7GMscf`
Address B	`myctaaJXJzqpegkfKTY6vxLm5JRhLqXBLc`
Address C	`msPZf6mnd1Rq41QJ5doaL9y8AFznznPzSg`

2.3 Transaction Workflow: A to B to C

Address A was funded with 1.0 BTC. The UTXO created at A was used as input to the transaction A to B. The UTXO created at B from that transaction was then used as input to B to C, forming a UTXO chain.

Funding TXID (coins sent to A)

```
b5156d78651e7f35d8db88ed1797b916d781a41eb592777cf1196eb1fc1d898e
```

TXID: Transaction A to B

```
884230236c9619a91fe92db5b4ec24ebe78a999d1b235a51a8ca3faba4fc6cdb
```

TXID: Transaction B to C

```
04ab98eda4d3944f457642cb2519db56f0865f1d33908dde0a3e718ef20c2d89
```

Chain verification: The UTXO at address B was confirmed to have txid equal to the A to B transaction. This was verified programmatically using `assert utxo_B["txid"] == txid_AB`.

2.4 Decoded Scripts – Transaction A to B

After creating and signing the raw transaction from A to B, `decoderawtransaction` was used to extract the scripts.

Challenge Script (scriptPubKey) for Address B

```
type: pubkeyhash
asm:  OP_DUP OP_HASH160 c691a89b835909af7affb1c7193e1e80c8d0875c
      OP_EQUALVERIFY OP_CHECKSIG
hex:  76a914c691a89b835909af7affb1c7193e1e80c8d0875c88ac
size: 25 bytes
```

This script **locks** the output to address B. To spend this output, the spender must provide a public key that hashes to `c691a89b835909af7affb1c7193e1e80c8d0875c` AND a valid signature from the corresponding private key.

2.5 Decoded Scripts – Transaction B to C

Challenge Script (scriptPubKey) for Address C

```
type: pubkeyhash
asm:  OP_DUP OP_HASH160 823bb37e51ca71b7bca3f3ae7cba4efa2f6852d1
      OP_EQUALVERIFY OP_CHECKSIG
hex:  76a914823bb37e51ca71b7bca3f3ae7cba4efa2f6852d188ac
size: 25 bytes
```

Response Script (scriptSig) – unlocking B's UTXO

```
asm:
  3044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0e6b30e800
  02203ea6e0dbcf4beae653ce8136049956a113526080332053f840cf1a3fffa60688[ALL]
  03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888

hex:
  473044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0e6b30e8
  0002203ea6e0dbcf4beae653ce8136049956a113526080332053f840cf1a3fffa60688012
  103eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888

size: ~107 bytes
  - DER signature: ~72 bytes (includes SIGHASH_ALL byte)
  - Compressed public key: 33 bytes
  - Push data opcodes: 2 bytes
```

Challenge-Response Match Verification

The scriptSig (response) satisfies the scriptPubKey (challenge) as follows:

Challenge requires	Response provides
hash160(pubKey) == c691a89b...	pubKey = 03eb7d9e... whose hash160 = c691a89b...
valid ECDSA signature	sig = 3044022010c1e9... verified against pubKey

2.6 Transaction Sizes – P2PKH

Transaction	Size (bytes)	vSize (vbytes)	Weight (WU)
A to B	191	191	764
B to C	191	191	764

For P2PKH: size = vsize = weight/4 because there is no witness data. All data counted at weight x4.

3 Part 1: btcdeb Script Execution (P2PKH)

3.1 Command Used

```
btcdeb --tx=<signed_hex_B_to_C> --txin=<signed_hex_A_to_B>
```

btcdeb loads the spending transaction (B to C) and the UTXO being spent (A to B), reconstructs the full script execution environment, and steps through each opcode showing the stack state.

3.2 Initial Script Load

```
btcdeb 5.0.24
LOG: signing segwit taproot
input tx index = 0; tx input vout = 0; value = 99990000
got witness stack of size 0
8 op script loaded.

Script:
  3044022010c1e9ad...  <- signature (from scriptSig)
  03eb7d9e163962...    <- public key (from scriptSig)
  <<< scriptPubKey >>>
  OP_DUP
  OP_HASH160
  c691a89b835909af7affb1c7193e1e80c8d0875c
  OP_EQUALVERIFY
  OP_CHECKSIG
```

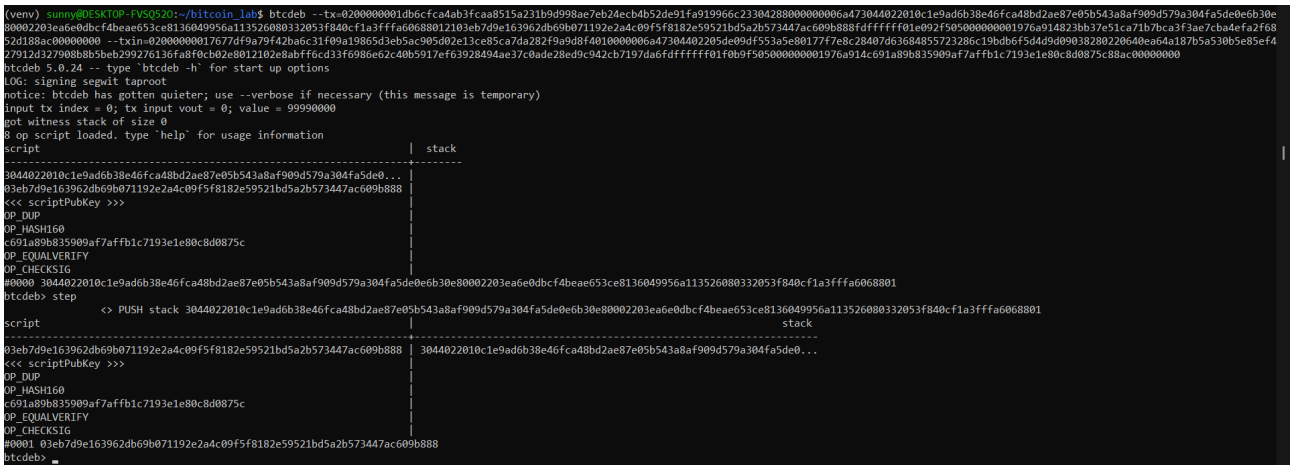

Figure 1: btcdeb: Initial script loaded – 8 opcodes, witness stack size 0 (P2PKH)

3.3 Step-by-Step Execution

Figure 2: Step 0-1: Signature and public key pushed onto stack from scriptSig

```
btcdeb> step
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_DUP                                                          | 03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888
OP_HASH160                                                      | 3044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0...
c691a89b835909af7affb1c7193e1e80c8d0875c                        |
OP_EQUALVERIFY                                                  |
OP_CHECKSIG                                                     |
#0003 OP_DUP
btcdeb>
```

Figure 3: Step 2 (OP_DUP): Public key duplicated – stack now has [pubKey, pubKey, sig]

```
btcdeb> step
                <> PUSH stack 03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_HASH160                                                      | 03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888
c691a89b835909af7affb1c7193e1e80c8d0875c                        | 03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888
OP_EQUALVERIFY                                                  | 3044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0...
OP_CHECKSIG                                                     |
#0004 OP_HASH160
btcdeb>
```

Figure 4: Step 3 (OP_HASH160): Top pubKey popped and hashed – hash160(pubKey) pushed

```
btcdeb> step
                <> POP  stack
                <> PUSH stack c691a89b835909af7affb1c7193e1e80c8d0875c
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
c691a89b835909af7affb1c7193e1e80c8d0875c                        |                          c691a89b835909af7affb1c7193e1e80c8d0875c
OP_EQUALVERIFY                                                  | 03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888
OP_CHECKSIG                                                     | 3044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0...
#0005 c691a89b835909af7affb1c7193e1e80c8d0875c
btcdeb>
```

Figure 5: Step 4: Expected hash c691a89b... pushed onto stack for comparison

```
btcdeb> step
                <> PUSH stack c691a89b835909af7affb1c7193e1e80c8d0875c
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_EQUALVERIFY                                                  |                          c691a89b835909af7affb1c7193e1e80c8d0875c
OP_CHECKSIG                                                     |                          c691a89b835909af7affb1c7193e1e80c8d0875c
                                                                | 03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888
                                                                | 3044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0...
#0006 OP_EQUALVERIFY
```

Figure 6: Step 5 (OP_EQUALVERIFY): Both hashes compared – equal, so both popped. Stack: [pubKey, sig]

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                                          |                                                                stack
----------------------------------------------------------------+----------------------------------------------------------------
OP_CHECKSIG                                                     | 03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888
                                                                | 3044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0...
#0007 OP_CHECKSIG
btcdeb>
```

Figure 7: Step 6: OP_CHECKSIG executing – ECDSA signature verification in progress

```
btcdeb> step
EvalChecksig() sigversion=0
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=0)
  sig         = 3044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0e6b30e80002203ea6e0dbcf4beae653ce8136049956a113526080332053f840cf1a3fffa6068801
  pub key     = 03eb7d9e163962db69b071192e2a4c09f5f8182e59521bd5a2b573447ac609b888
  script code = 76a914c691a89b835909af7affb1c7193e1e80c8d0875c88ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=99990000)
- sigversion = SIGVERSION_BASE (non-segwit style)
 << txTo.vin[nInput=0].prevout = COutPoint(884230236c, 0)
(SerializeScriptCode)
 << scriptCode.size()=25 - nCodeSeparators=0
 << script:76a914c691a89b835909af7affb1c7193e1e80c8d0875c88ac
 << txTo.vin[nInput].nSequence = 4294967293 [0xfffffffd]
  sighash     = 3183e9a36a27c4c60922dcbff3a48c7509a8b00cfa8e83a8b0a69747950eed78
  pubkey.VerifyECDSASignature(sig=3044022010c1e9ad6b38e46fca48bd2ae87e05b543a8af909d579a304fa5de0e6b30e80002203ea6e0dbcf4beae653ce8136049956a113526080332053f840cf1a3fffa60688, sighash=3183e9a36a27c4c60922dcbff3a
48c7509a8b00cfa8e83a8b0a69747950eed78):
  result: success
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                                          |                                                       stack
----------------------------------------------------------------+-------------------------------------------------------------
                                                                |                                                          01
btcdeb>
```

Figure 8: Step 7 (OP_CHECKSIG): result: success – sigversion=SIGVERSION_BASE – stack: [01] = TRUE

3.4 Final Result

OP_CHECKSIG result: success
sigversion = SIGVERSION_BASE (non-segwit style)
Final stack: `[01]` = TRUE
Transaction is VALID

4 Part 2: P2SH-P2WPKH SegWit Transactions

4.1 Generated SegWit Addresses

Three P2SH-SegWit addresses were generated using `getnewaddress "label" "p2sh-segwit"`:

Label	Address
Address A'	`2MyZY3tPj7pdSSdy6VheZrdHgwJiaSfBBoJ`
Address B'	`2N2TwC36qw1AK8CuaT5sKVqXgddVe6efFX9`
Address C'	`2MvmgbmncoLMMrVWUUG1qYbcufrHKSK4hX8`

4.2 Transaction Workflow: A' to B' to C'

TXID: Transaction A' to B'

```
c4f98998f533a3922dd28f45f76e9f40e0b0c97e90cf918ebbbfc0d6314df03b
```

TXID: Transaction B' to C'

```
6b6d287fdc517b57a30bc92b8017116d8dd7695ead4b14db26a17f5d9c535a4d
```

4.3 Decoded Scripts – Transaction A' to B'

Challenge Script (scriptPubKey) for Address B'

```
type: scripthash
asm:  OP_HASH160 652084904b2946273d6bbf74de7309344c682426 OP_EQUAL
hex:  a914652084904b2946273d6bbf74de7309344c68242687
size: 23 bytes
```

This P2SH script commits only to the **hash of the redeemScript**, not the full script. The actual redeemScript is only revealed when spending. This is shorter and hides script complexity from the blockchain.

4.4 Decoded Scripts – Transaction B' to C'

Challenge Script (scriptPubKey) for Address C'

```
type: scripthash
asm:  OP_HASH160 26ab12185562333f76775557bf3ad89ff594f839 OP_EQUAL
hex:  a91426ab12185562333f76775557bf3ad89ff594f83987
size: 23 bytes
```

Response Script (scriptSig) – spending B's UTXO

```
asm:  001428b4428962cc1d516dd6bede9760c751e1eef38d
hex:  16001428b4428962cc1d516dd6bede9760c751e1eef38d
size: 23 bytes only

Decoded redeemScript: OP_0 <28b4428962cc1d516dd6bede9760c751e1eef38d>
  = OP_0 <hash160(publicKey)>  <- this is the P2WPKH witness program
```

Key difference from P2PKH: The scriptSig contains ONLY the 23-byte redeemScript. The signature and public key are NOT here – they are in the witness field.

Witness Data (segregated)

```
[0] signature:
    304402207bf59e890560336cd796e95fe4d3b52f8ffc698cc9ab2ec50a5ee1c5a195411f
    0220515f1916e27816bff55c32b41bf7cafac8635470a87f6ef63a403830da3fff6701
    size: ~72 bytes -- counted at weight x1 (discounted)

[1] public key:
    0211b45cffff1f601467a300ad1bd1f657a101633a645bc4e58466112e75c3e06e
    size: 33 bytes -- counted at weight x1 (discounted)
```

Challenge-Response Match Verification

Challenge (scriptPubKey)	Response (scriptSig + witness)
OP_HASH160 652084... OP_EQUAL	scriptSig pushes redeemScript hash160(redeemScript) must == 652084... witness[1] pubkey hashed must match OP_0 program witness[0] signature verified against pubkey

4.5 Transaction Sizes – SegWit

Transaction	Size (bytes)	vSize (vbytes)	Weight (WU)
A' to B'	215	134	533
B' to C'	215	134	533

Note: Raw size (215) is larger than P2PKH (191) because it includes witness data. But vsize (134) is much smaller than P2PKH (191) because witness bytes get 75% discount.

5 Part 2: btcdeb Script Execution (SegWit)

5.1 Command Used

```
btcdeb --tx=<signed_hex_B_prime_to_C_prime>
       --txin=<signed_hex_A_prime_to_B_prime>
```

5.2 Startup Output

```
btcdeb 5.0.24
LOG: signing segwit taproot
input tx index = 0; tx input vout = 0; value = 99990000
got witness stack of size 2
script sig non-empty; embedded P2SH (extracting payload)
hash source = 001428b4428962cc1d516dd6bede9760c751e1eef38d
22 bytes (P2WPKH)
valid script
5 op script loaded.
```

Key observations: btcdeb detected **witness stack of size 2** (signature + pubkey), extracted the embedded P2SH payload, identified it as P2WPKH (22 bytes), and loaded only 5 opcodes – the inner witness script, not the P2SH wrapper.

Figure 9: btcdeb: SegWit initial load – witness stack size 2, P2WPKH extracted, 5 ops loaded

5.3 Script Loaded

```
Stack pre-loaded from witness:
  [top]: 0211b45cffff1f601467a300...  (public key)
  [  1]: 304402207bf59e890560336c...  (signature)

Opcodes:
  OP_DUP
  OP_HASH160
  28b4428962cc1d516dd6bede9760c751e1eef38d
  OP_EQUALVERIFY
  OP_CHECKSIG
```

5.4 Step-by-Step Execution

```
btcdeb> step
                <> PUSH stack 0211b45cffff1f601467a300ad1bd1f657a101633a645bc4e58466112e75c3e06e
script                                  |                                                       stack
----------------------------------------+-------------------------------------------------------
OP_HASH160                              | 0211b45cffff1f601467a300ad1bd1f657a101633a645bc4e58466112e75c3e06e
28b4428962cc1d516dd6bede9760c751e1eef38d | 0211b45cffff1f601467a300ad1bd1f657a101633a645bc4e58466112e75c3e06e
OP_EQUALVERIFY                          | 304402207bf59e890560336cd796e95fe4d3b52f8ffc698cc9ab2ec50a5ee1c...
OP_CHECKSIG                             |
#0001 OP_HASH160
btcdeb>
```

Figure 10: Step 0 (OP_DUP): Public key duplicated – stack: [pubKey, pubKey, sig]

```
btcdeb> step
                <> POP  stack
                <> PUSH stack 28b4428962cc1d516dd6bede9760c751e1eef38d
script                                  |                                                       stack
----------------------------------------+-------------------------------------------------------
28b4428962cc1d516dd6bede9760c751e1eef38d |                                28b4428962cc1d516dd6bede9760c751e1eef38d
OP_EQUALVERIFY                          | 0211b45cffff1f601467a300ad1bd1f657a101633a645bc4e58466112e75c3e06e
OP_CHECKSIG                             | 304402207bf59e890560336cd796e95fe4d3b52f8ffc698cc9ab2ec50a5ee1c...
#0002 28b4428962cc1d516dd6bede9760c751e1eef38d
btcdeb>
```

Figure 11: Step 1 (OP_HASH160): pubKey hashed – hash160(pubKey) pushed onto stack

```
btcdeb> step
                <> PUSH stack 28b4428962cc1d516dd6bede9760c751e1eef38d
script                                  |                                                       stack
----------------------------------------+-------------------------------------------------------
OP_EQUALVERIFY                          |                                28b4428962cc1d516dd6bede9760c751e1eef38d
OP_CHECKSIG                             |                                28b4428962cc1d516dd6bede9760c751e1eef38d
                                        | 0211b45cffff1f601467a300ad1bd1f657a101633a645bc4e58466112e75c3e06e
                                        | 304402207bf59e890560336cd796e95fe4d3b52f8ffc698cc9ab2ec50a5ee1c...
#0003 OP_EQUALVERIFY
btcdeb>
```

Figure 12: Step 2: Expected hash 28b4428962... pushed for comparison with computed hash

```
btcdeb> step
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
                <> POP  stack
script                                  |                                                       stack
----------------------------------------+-------------------------------------------------------
OP_CHECKSIG                             | 0211b45cffff1f601467a300ad1bd1f657a101633a645bc4e58466112e75c3e06e
                                        | 304402207bf59e890560336cd796e95fe4d3b52f8ffc698cc9ab2ec50a5ee1c...
#0004 OP_CHECKSIG
btcdeb>
```

Figure 13: Step 3 (OP_EQUALVERIFY): Both hashes match – verified. Stack: [pubKey, sig]

```
btcdeb> step
EvalChecksig() sigversion=1
Eval Checksig Pre-Tapscript
GenericTransactionSignatureChecker::CheckECDSASignature(71 len sig, 33 len pubkey, sigversion=1)
  sig         = 304402207bf59e890560336cd796e95fe4d3b52f8ffc698cc9ab2ec50a5ee1c5a195411f0220515f1916e27816bff55c32b41bf7cafac8635470a87f6ef63a403830da3fff6701
  pub key     = 0211b45cffff1f601467a300ad1bd1f657a101633a645bc4e58466112e75c3e06e
  script code = 76a91428b4428962cc1d516dd6bede9760c751e1eef38d88ac
  hash type   = 01 (SIGHASH_ALL)
SignatureHash(nIn=0, nHashType=01, amount=99990000)
- sigversion == SIGVERSION_WITNESS_V0
  sighash    = c33eb805e66904d09f1949d5ec5645d2245dac722360754f1f557c495a865337
  pubkey.VerifyECDSASignature(sig=304402207bf59e890560336cd796e95fe4d3b52f8ffc698cc9ab2ec50a5ee1c5a195411f0220515f1916e27816bff55c32b41bf7cafac8635470a87f6ef63a403830da3fff67, sighash=c33eb805e66904d09f1949d5ec5
645d2245dac722360754f1f557c495a865337):
  result: success
                <> POP  stack
                <> POP  stack
                <> PUSH stack 01
script                                  |                                                       stack
----------------------------------------+-------------------------------------------------------
                                        |                                                          01
btcdeb>
```

Figure 14: Step 4 (OP_CHECKSIG): sigversion=SIGVERSION_WITNESS_V0 – result: success – stack: [01]

5.5 Final Result

OP_CHECKSIG result: success
sigversion = SIGVERSION_WITNESS_V0 (BIP 143 sighash)
Final stack: `[01]` = TRUE
Transaction is VALID

Critical difference from P2PKH: The sigversion was `SIGVERSION_WITNESS_V0`, meaning the signature hash was computed using the BIP 143 algorithm (signs the input value explicitly, prevents malleability). The witness data was pre-loaded onto the stack before opcode execution, rather than being pushed via scriptSig.

6 Part 3: Comparative Analysis

6.1 Transaction Size Comparison

All values measured from actual transactions broadcast on the regtest network:

Metric	P2PKH (B to C)	SegWit (B' to C')	Difference	Savings
Size (bytes)	191	215	+24	SegWit is larger raw
vSize (vbytes)	191	134	-57	29.8% smaller
Weight (WU)	764	533	-231	30.2% smaller

The raw byte size of SegWit is larger because it includes witness data bytes. However, vsize is significantly smaller because witness bytes are counted at only 1/4 weight.

6.2 Script Structure Comparison

Component	P2PKH (Legacy)	P2SH-P2WPKH (SegWit)
scriptPubKey type scriptPubKey size scriptPubKey	pubkeyhash 25 bytes OP_DUP OP_HASH160 ¡20B hash¿ OP_EQUALVERIFY OP_CHECKSIG	scripthash 23 bytes OP_HASH160 ¡20B hash¿ OP_EQUAL
scriptSig content scriptSig size	signature + pubkey 107 bytes	redeemScript only 23 bytes
Witness Witness size	none 0 bytes	signature + pubkey 107 bytes
sigversion Sighash algorithm	SIGVERSION_BASE Legacy double SHA256	SIGVERSION_WITNESS_V0 BIP 143 (signs value)

6.3 Weight Calculation – Worked Example

P2PKH (191 bytes, no witness)

```
All bytes are non-witness:
weight = 191 x 4 = 764 WU
vbytes = 764 / 4 = 191 vbytes
```

P2SH-P2WPKH (215 bytes total)

```
Non-witness bytes: ~108 bytes (version, inputs without sig, outputs, locktime)
Witness bytes:     ~107 bytes (signature ~72B + pubkey 33B + stack items)

weight = (108 x 4) + (107 x 1)
       = 432 + 107
       = ~533 WU   (actual measured: 533)

vbytes = ceil(533 / 4) = 134 vbytes
```

The witness discount directly explains why 215 raw bytes becomes only 134 vbytes.

6.4 Why SegWit Transactions Are Smaller

Reason 1: Witness Data Weight Discount

Bitcoin's weight unit system assigns different costs to different transaction parts:

- **Non-witness data** (version, inputs, outputs, locktime): counted at weight **x4**

- **Witness data** (signatures, pubkeys): counted at weight **x1** (75% discount)

Signatures and public keys are the largest components of most transactions (105 bytes). By moving them to the witness field, their contribution to the weight calculation drops by 75%.

Reason 2: Smaller scriptSig

Part	P2PKH weight	SegWit weight
scriptSig (107 bytes vs 23 bytes)	107 x 4 = 428 WU	23 x 4 = 92 WU
witness (0 bytes vs 107 bytes)	0 x 1 = 0 WU	107 x 1 = 107 WU
Total for sig+pubkey part	**428 WU**	**199 WU**

Reason 3: Fee Benefit

```
Bitcoin fees = fee_rate (sat/vbyte) x vbytes

At 10 sat/vbyte:
  P2PKH:  191 vbytes x 10 = 1910 satoshis
  SegWit: 134 vbytes x 10 = 1340 satoshis
  Saving: 570 satoshis per transaction (~30%)
```

Reason 4: Block Throughput

Bitcoin's block limit is 4,000,000 weight units. Because SegWit transactions use fewer weight units per transaction, more transactions fit per block, increasing network throughput without a hard fork.

Reason 5: Transaction Malleability Fix

In P2PKH, signatures are inside scriptSig which is part of the txid calculation. A third party could alter the signature (without invalidating it) to produce a different txid – this is **transaction malleability**. In SegWit, signatures are in the witness field which is NOT part of the txid calculation, eliminating malleability and enabling second-layer protocols like the Lightning Network.

7 Conclusion

This lab successfully implemented Legacy P2PKH and SegWit P2SH-P2WPKH Bitcoin transactions on a local regtest network using Python and bitcoind RPC. All transactions were created, signed, broadcast, confirmed, and verified using btcdeb.

Key findings:

- The UTXO chain was correctly formed: txid `884230236c...` (A to B) became the input for transaction B to C, and txid `c4f98998f5...` (A' to B') became the input for B' to C'.
- P2PKH challenge scripts use 25 bytes with `OP_DUP OP_HASH160 ... OP_EQUALVERIFY OP_CHECKSIG`. SegWit challenge scripts use only 23 bytes with `OP_HASH160 ... OP_EQUAL`.
- P2PKH scriptSig is 107 bytes (signature + pubkey). SegWit scriptSig is only 23 bytes (redeemScript); the 107 bytes move to witness.
- btcdeb confirmed `result: success` for both P2PKH (sigversion=BASE) and SegWit (sigversion=WITNESS_V0).
- Real measured savings: 191 vbytes (P2PKH) vs 134 vbytes (SegWit) = 29.8% reduction.
- SegWit uses BIP 143 sighash which covers input value, preventing amount manipulation attacks.

Repository: https://github.com/Sanskriti-hello/bitcoin-transaction-lab

Team HighOnByte | CS 216 | March 2026